

WEEK 4 - TASK 12 (EH)

ETHICAL HACKING AND WEB APPLICATION SECURITY ASSESSMENT

Submitted By	MANOJ
Role	Cyber Security Intern
Internship Organisation	TechBiz Security
Internship ID	TBS-INT-2026-518
Submitted Date	14 September 2026

Controlled Laboratory / Educational Security Assessment

DVWA and TryHackMe OWASP Top 10 2025 learning activities

1. TABLE OF CONTENTS

Section	Title
2	Introduction
3	Objectives
4	Lab Environment
5	Tools and Platforms
6	Practical Activity 1 – Reflected Cross-Site Scripting (XSS)
7	Practical Activity 2 – SQL Injection
8	Practical Activity 3 – Command Injection
9	Practical Activity 4 – Stored Cross-Site Scripting
10	Practical Activity 5 – Brute Force
11	Practical Activity 6 – File Inclusion
12	Practical Activity 7 – Cryptographic Failures
13	OWASP Top 10 2025 Learning Activity
14	Security Observations
15	Mitigation and Security Recommendations
16	Ethical Hacking and Security Best Practices
17	Learning Outcomes
18	Conclusion
19	Evidence Summary

2. INTRODUCTION

The purpose of this Week 4 Ethical Hacking task is to develop practical understanding of common web application security weaknesses through controlled educational exercises. The evidence provided for this report consists primarily of demonstrations in Damn Vulnerable Web Application (DVWA), together with a completed TryHackMe learning activity covering selected OWASP Top 10 2025 concepts.

The practical activities cover reflected and stored Cross-Site Scripting (XSS), SQL Injection, Command Injection, Brute Force authentication testing, File Inclusion, and a Cryptographic Failures exercise. These activities illustrate how insecure input handling, weak security controls, and insecure application design can create attack surfaces.

In ethical hacking and penetration testing, controlled demonstrations such as these help security practitioners recognize vulnerable application behavior, understand attack categories, collect evidence, and communicate security observations and defensive measures. The activities documented here are laboratory exercises and are not presented as testing of real-world systems.

3. OBJECTIVES

- Understand common web application vulnerabilities.
- Practically demonstrate vulnerabilities in a controlled DVWA environment.
- Understand the security impact of input validation and improper security controls.
- Practice identifying vulnerabilities through ethical hacking techniques.
- Understand common attack categories related to web application security.
- Improve practical penetration testing and vulnerability assessment skills.
- Relate practical findings to OWASP web application security concepts.
- Document technical activities professionally for cybersecurity reporting.

4. LAB ENVIRONMENT

The screenshots show a Kali Linux environment running a local DVWA instance through a web browser. The DVWA pages are accessed through a localhost-based laboratory URL. A separate screenshot shows a TryHackMe learning environment for the OWASP Top 10 2025: IAAA Failures activity.

- Kali Linux
- Damn Vulnerable Web Application (DVWA)
- Web browser
- Localhost-based laboratory environment
- TryHackMe OWASP Top 10 2025 learning environment

The evidence supports describing these activities as controlled learning exercises. No claim is made about a production or third-party target.

5. TOOLS AND PLATFORMS

Tool / Platform	Purpose
Kali Linux	Security testing and penetration testing environment
DVWA	Deliberately vulnerable web application used for security testing practice
Web Browser	Interaction with the vulnerable web application
TryHackMe	Cybersecurity learning and practical security training platform

6. PRACTICAL ACTIVITY 1 - REFLECTED CROSS-SITE SCRIPTING (XSS)

Technical explanation: Reflected Cross-Site Scripting occurs when user-controlled input is returned in a web response without appropriate context-aware output encoding or validation. If the browser interprets the reflected input as executable JavaScript, an attacker may be able to perform browser-side actions within the application's security context.

The provided DVWA screenshot demonstrates the Reflected XSS page with a script payload entered into the name parameter. The evidence shows the payload present in the input field and the application response area below it. This demonstrates the vulnerable input-reflection behavior in the controlled DVWA environment.

Security relevance: Depending on application context, reflected XSS can potentially contribute to session-related attacks, phishing, malicious redirection, or unauthorized browser-side actions. These are potential impacts and are not claimed as demonstrated results in this evidence.

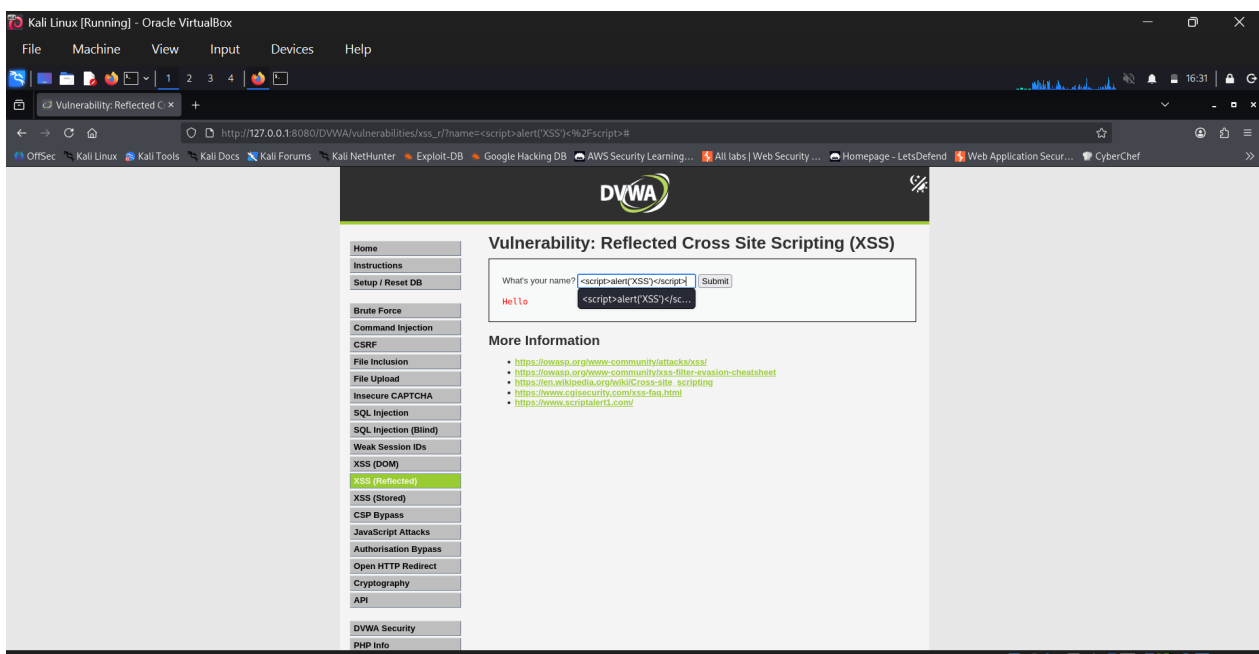


Figure 1: Reflected Cross-Site Scripting Demonstration in DVWA

7. PRACTICAL ACTIVITY 2 - SQL INJECTION

Technical explanation: SQL Injection occurs when untrusted input is incorporated into a database query in an unsafe manner. This can allow the structure or logic of a query to be influenced by the supplied input.

The DVWA screenshot shows an SQL injection input supplied to the User ID field. After submission, the application returns database information including an ID, first name, and surname. This is evidence of unsafe handling of the supplied input in the laboratory application.

Security relevance: SQL Injection can potentially lead to unauthorized database access, authentication bypass, data disclosure, data manipulation, or further escalation depending on the application's database configuration and privileges. The screenshot does not by itself establish extraction of additional database records beyond the displayed response.

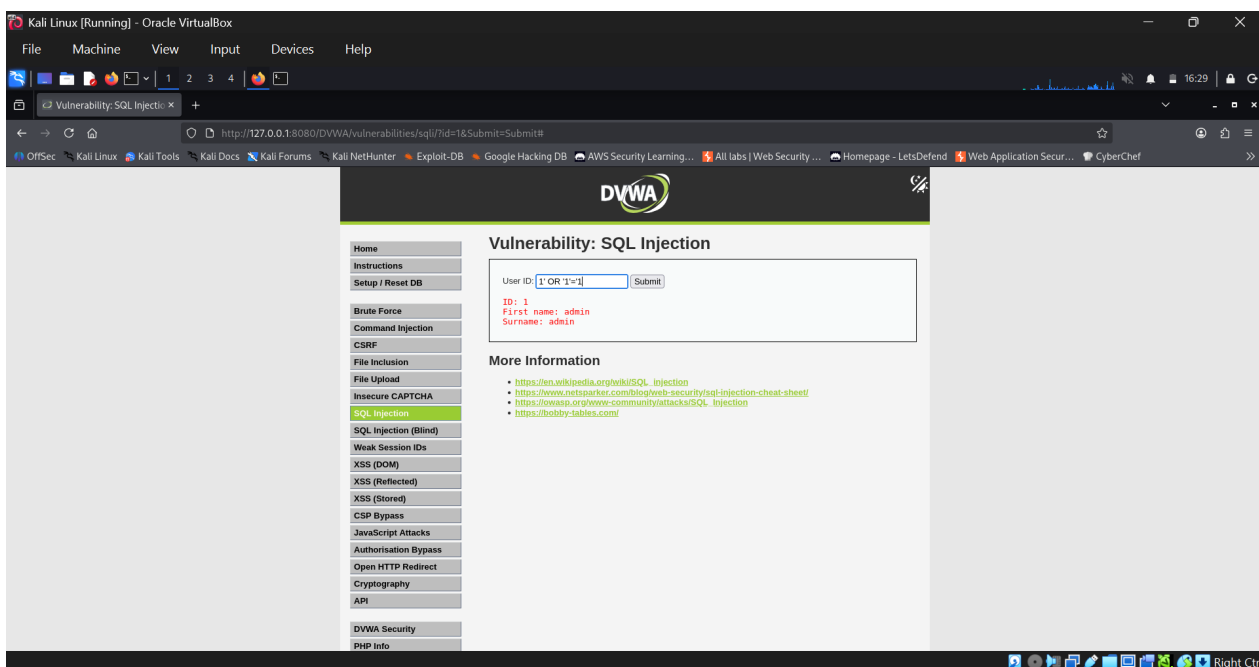


Figure 2: SQL Injection Demonstration in DVWA

8. PRACTICAL ACTIVITY 3 - COMMAND INJECTION

Technical explanation: Command Injection occurs when untrusted input is incorporated into operating-system commands without appropriate validation or safe command execution. A vulnerable application may unintentionally allow user input to influence a system command.

The provided DVWA Command Injection screenshot shows the 'Ping a device' functionality with an IP address supplied to the application. The resulting output displays the ping operation and its response statistics. This demonstrates the application's interaction with an operating-system command in the controlled laboratory.

Security relevance: If command injection is present, potential impacts include unauthorized operating-system command execution, information disclosure, system compromise, and possible privilege escalation depending on the privileges of the application process. Arbitrary command execution is not claimed here because it is not visibly demonstrated in the evidence.

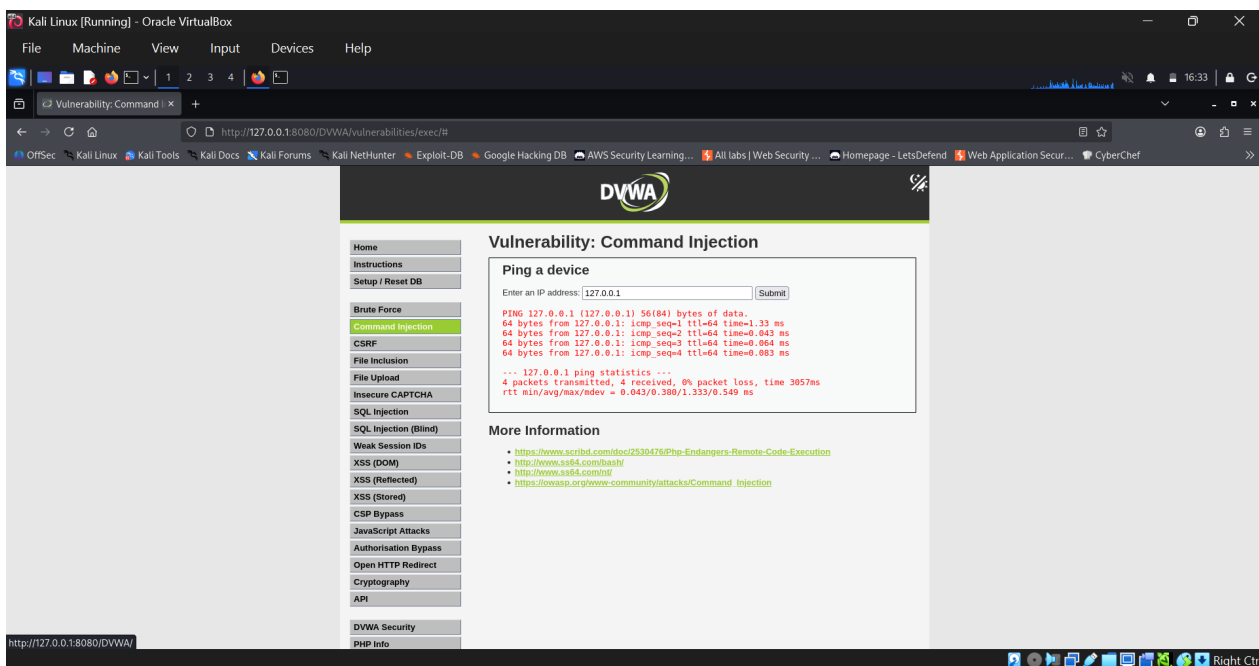


Figure 3: Command Injection Demonstration in DVWA

9. PRACTICAL ACTIVITY 4 - STORED CROSS-SITE SCRIPTING

Technical explanation: Stored XSS occurs when application-controlled storage retains attacker-controlled content and later displays that content to users without adequate output encoding.

The provided DVWA screenshot shows the Stored XSS guestbook functionality and previously submitted entries. The page contains Name and Message fields and displays existing guestbook content, including entries containing encoded script-like content. The screenshot therefore demonstrates the stored-content functionality and the presence of previously submitted test entries.

Reflected vs. Stored XSS: Reflected XSS is returned immediately in the response to a request, whereas Stored XSS is retained by the application and may be rendered when affected content is subsequently viewed. Stored XSS can have broader exposure when stored content is presented to multiple users.

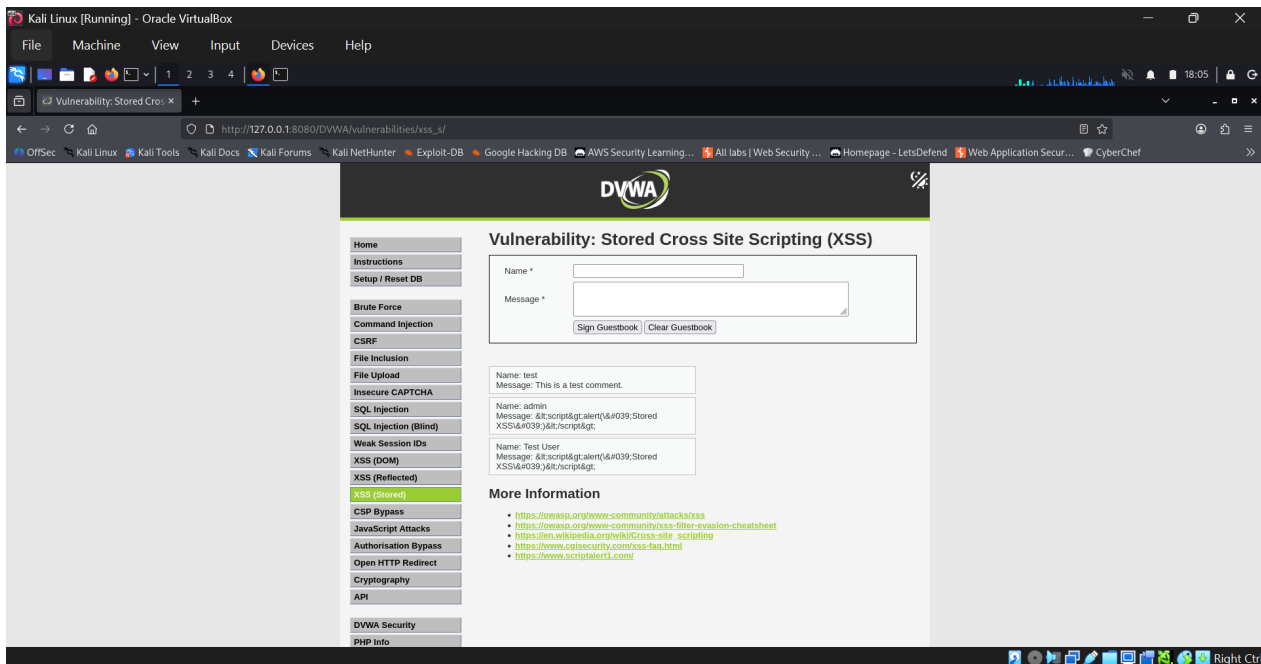


Figure 4: Stored Cross-Site Scripting Functionality in DVWA

10. PRACTICAL ACTIVITY 5 - BRUTE FORCE

Technical explanation: A Brute Force attack repeatedly attempts authentication credentials in an effort to discover valid access. Effective defenses make repeated guessing difficult through controls such as rate limiting, lockout or progressive delays, strong passwords, and multi-factor authentication.

The DVWA Brute Force screenshot shows an incorrect username/password attempt producing an authentication failure message. The displayed message also states that an account may be locked after too many failed logins and instructs the user to try again after a specified period. This is the visible evidence from the exercise.

Security relevance: Authentication controls are important in reducing credential-guessing risk. Security monitoring should also identify repeated authentication failures and suspicious login patterns.

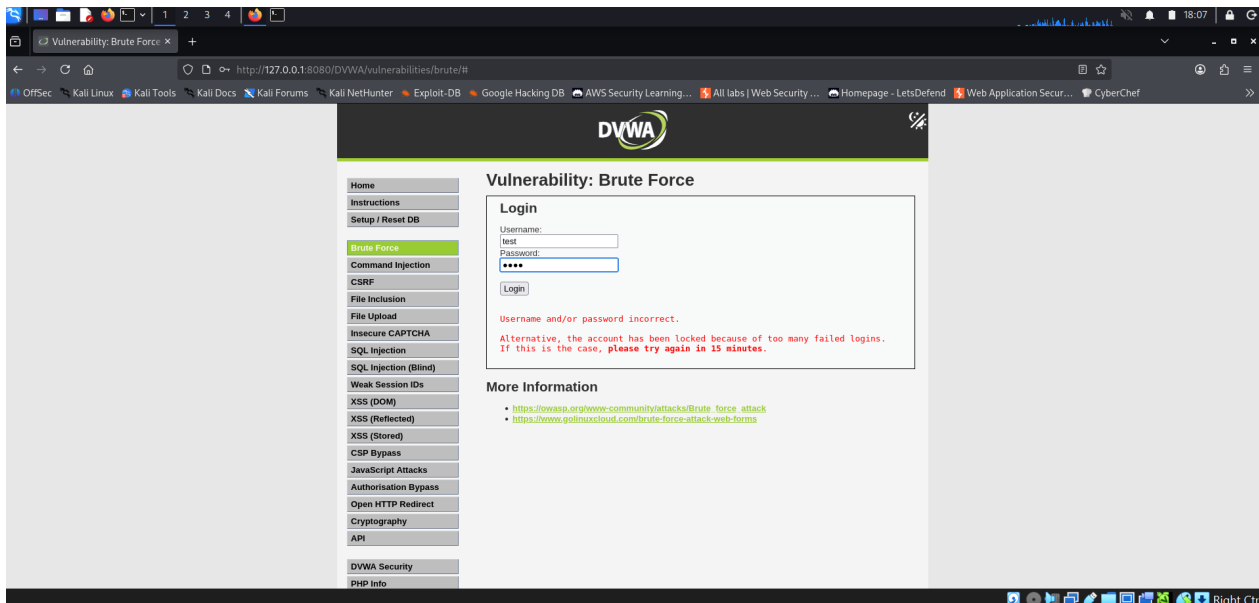


Figure 5: Brute Force Authentication Testing in DVWA

11. PRACTICAL ACTIVITY 6 - FILE INCLUSION

Technical explanation: File Inclusion vulnerabilities arise when an application dynamically includes files based on unsafe user-controlled input. Local File Inclusion (LFI) generally concerns inclusion of files available on the local system, while Remote File Inclusion (RFI) involves attempting to include content from a remote location.

The DVWA File Inclusion screenshot shows the laboratory page and an explicit message that remote inclusion depends on PHP configuration. It also states that the relevant `allow_url_include` functionality is not enabled in the displayed environment. The page provides links to local files such as `file1.php`, `file2.php`, and `file3.php`.

Security relevance: File Inclusion weaknesses can contribute to unauthorized file disclosure, sensitive information exposure, and, in vulnerable configurations, potential code execution. The screenshot does not demonstrate successful remote file inclusion; in fact, it explicitly indicates that the relevant PHP functionality is disabled.

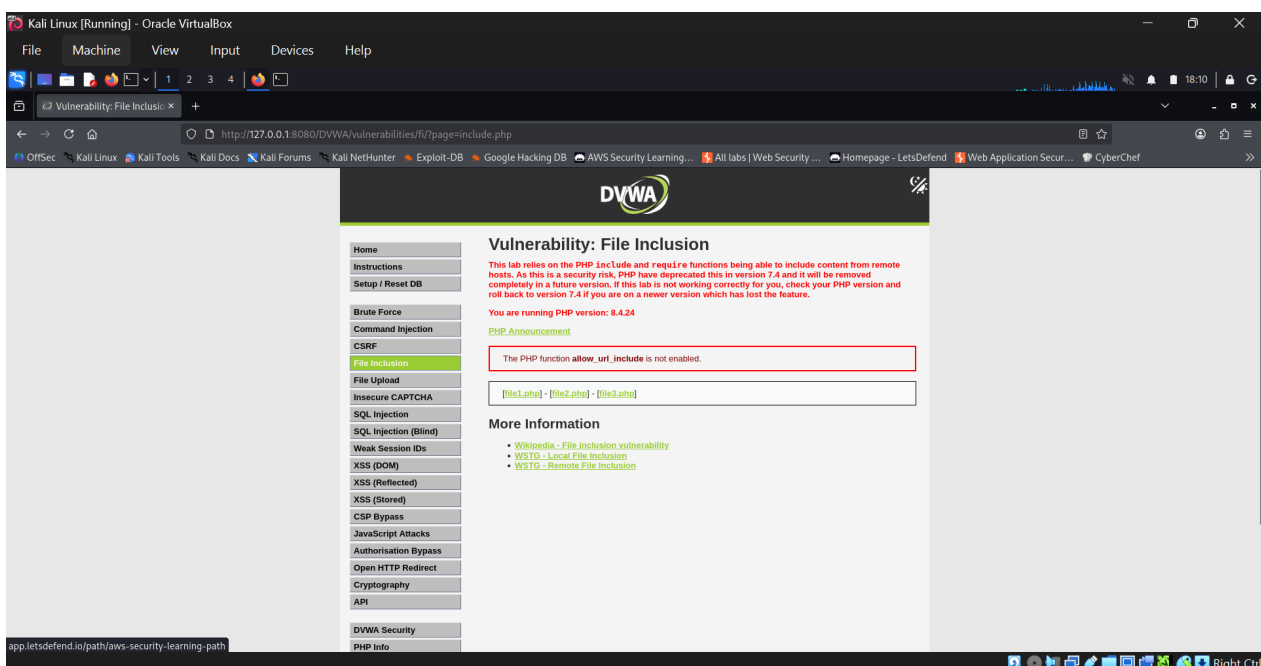


Figure 6: File Inclusion Testing in DVWA

12. PRACTICAL ACTIVITY 7 - CRYPTOGRAPHIC FAILURES

Technical explanation: Cryptographic Failures refer to weaknesses in the protection of sensitive information through cryptography, including insecure algorithms, poor key management, weak randomness, insecure cipher modes, improper padding, or exposure of sensitive tokens.

The DVWA Cryptography Problems page shown in the screenshot presents a token-based exercise. The page indicates that a token has been obtained and provides a token field for the laboratory exercise. The screenshot also lists learning references related to topics such as AES-ECB padding, padding oracle attacks, XOR encryption, and PKCS#7 padding.

Security relevance: Secure cryptographic implementation is important for protecting authentication tokens, confidential information, session information, and other sensitive application data. The screenshot alone does not establish that a specific cryptographic attack was successfully completed.

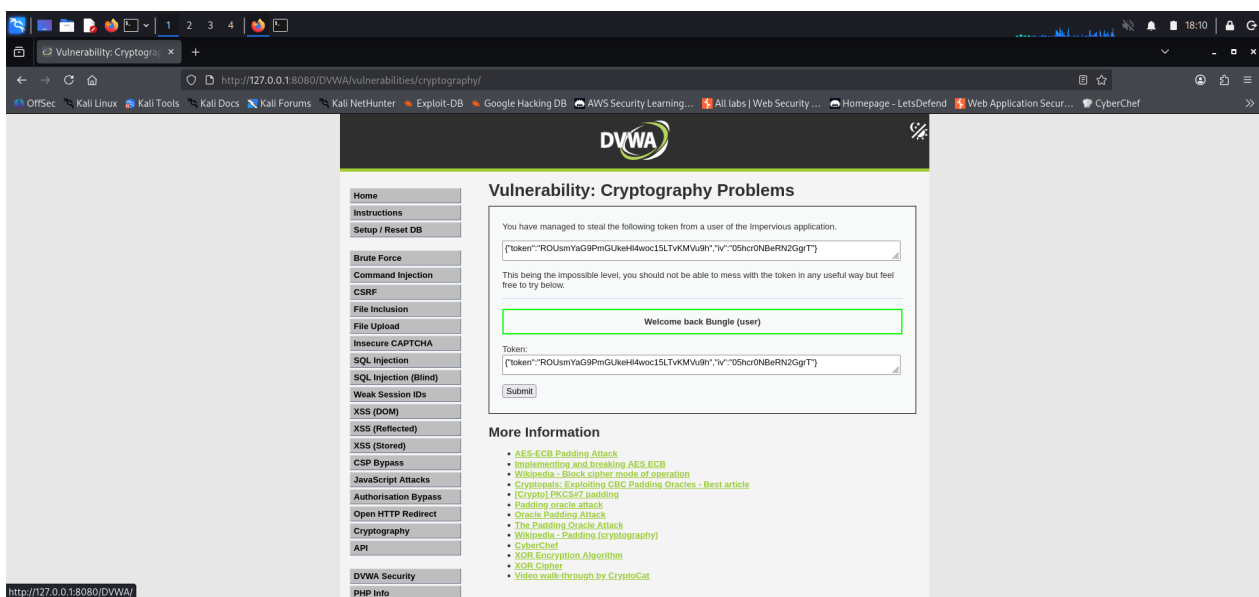


Figure 7: Cryptographic Security Exercise in DVWA

13. OWASP TOP 10 2025 LEARNING ACTIVITY

The OWASP Top 10 is widely used as a reference for understanding important categories of web application security risk. The attached TryHackMe screenshot shows completion of the learning activity titled OWASP Top 10 2025: IAAA Failures. The screenshot indicates that the room was completed at 100%.

The visible task list includes concepts related to Broken Access Control, Authentication Failures, and Logging and Alerting Failures. These areas are important during security assessments because access control, authentication, and security monitoring directly influence whether unauthorized actions can occur and whether suspicious activity can be detected and investigated.

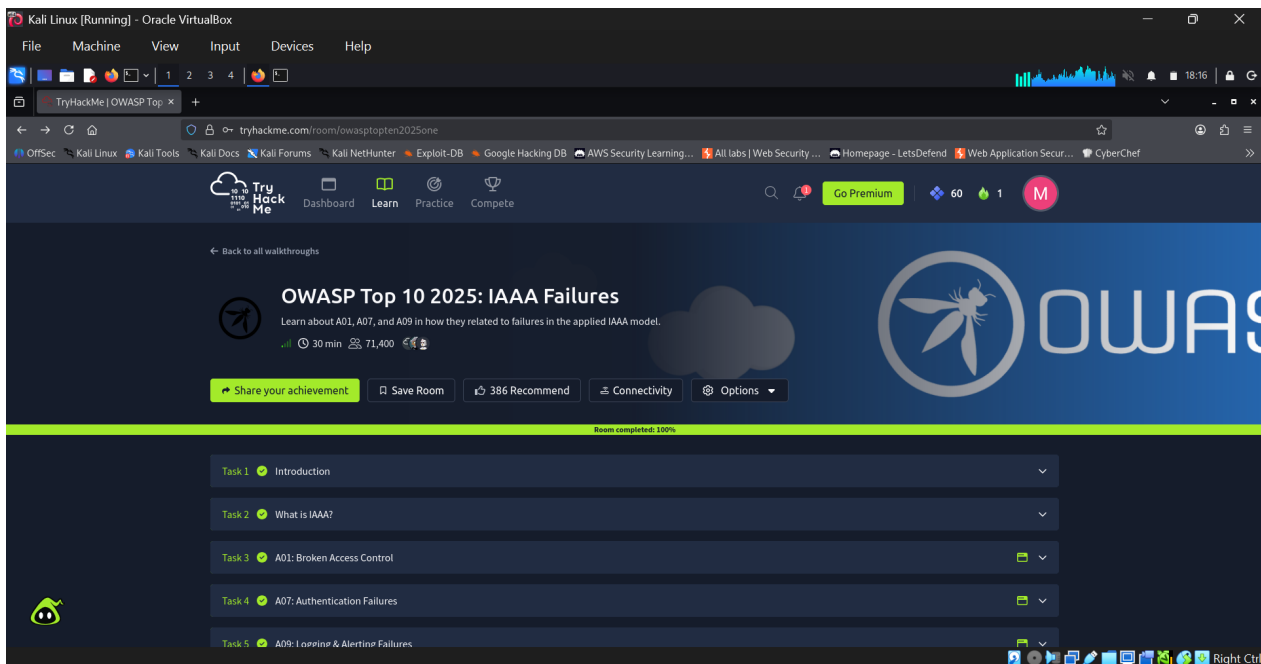


Figure 8: TryHackMe OWASP Top 10 2025 Learning Activity Completion

14. SECURITY OBSERVATIONS

No.	Security Area	Observation	Security Importance
1	Input Validation	Multiple DVWA exercises demonstrate risks associated with untrusted input exposing application functionality to input injection attacks.	Input validation is a critical security control that prevents malicious input from being processed by the application.
2	Authentication Security	Brute force testing demonstrates the importance of limiting login attempts, strong credentials, MFA, and monitoring for suspicious activity.	Robust authentication mechanisms are essential for verifying the identity of users and preventing unauthorized access.
3	Web Application Security	XSS and SQL Injection demonstrate common web application vulnerabilities. database input can affect confidentiality and integrity of data.	Web application security is a top priority as these vulnerabilities can lead to data breaches and service disruption.
4	Operating System Interaction	Command Injection demonstrates risks when web applications interact with the operating system.	Secure interaction with the operating system is crucial to prevent privilege escalation and system compromise.
5	File Handling	File Inclusion demonstrates risks associated with improper file inclusion controls that may expose local files or, in vulnerable versions, allow remote code execution.	Proper file handling and inclusion controls are necessary to protect sensitive system files and prevent remote execution.
6	Cryptography	Cryptographic exercises demonstrate the importance of using strong, modern encryption algorithms and secure key management.	Strong cryptography is fundamental for protecting sensitive data in transit and at rest.
7	Security Monitoring	OWASP learning content highlights the importance of logging and alerting for detecting and responding to security incidents.	Effective logging and alerting are critical for incident response and continuous security improvement.

15. MITIGATION AND SECURITY RECOMMENDATIONS

For XSS: Apply context-aware output encoding; validate input; use an appropriate Content Security Policy; and avoid inserting untrusted input directly into HTML or JavaScript contexts.

For SQL Injection: Use parameterized queries or prepared statements; avoid building SQL through string concatenation; apply strict validation; and use least-privilege database accounts.

For Command Injection: Avoid passing user input directly to operating-system commands; use safe APIs instead of shell execution where possible; apply allow-list validation; and run applications with least privilege.

For Brute Force: Implement rate limiting; use account lockout or progressive delays appropriately; enable multi-factor authentication; monitor repeated authentication failures; and enforce strong password policies.

For File Inclusion: Avoid dynamic file inclusion based directly on user input; use allow-lists for permitted files; disable unnecessary remote inclusion functionality; and apply secure server-side configuration.

For Cryptographic Failures: Use modern, well-reviewed cryptographic algorithms; protect encryption keys; use secure random number generation; protect authentication/session tokens; and avoid deprecated or insecure cryptographic configurations.

For Logging and Alerting: Implement centralized logging; monitor authentication failures; create alerts for suspicious activity; protect log integrity and access; and establish appropriate retention and monitoring procedures.

16. ETHICAL HACKING AND SECURITY BEST PRACTICES

All security testing should be performed only against systems for which explicit authorization has been obtained. Ethical hacking requires clear boundaries so that testing remains lawful, controlled, and focused on the agreed security objectives.

- Authorization: Obtain explicit permission before testing.
- Scope definition: Identify systems, applications, accounts, and testing boundaries.
- Responsible testing: Avoid unnecessary disruption and destructive activity.
- Data protection: Protect sensitive information encountered during testing.
- Evidence collection: Capture accurate, relevant evidence without fabricating results.
- Professional reporting: Document methods, observations, limitations, and recommendations clearly.
- Responsible disclosure: Communicate identified security weaknesses through the appropriate authorized channels.

17. LEARNING OUTCOMES

The activities provided practical exposure to common web application vulnerabilities and security concepts in a controlled DVWA environment. The exercises improved recognition of vulnerable application behavior and provided hands-on context for XSS, SQL Injection, Command Injection, authentication testing, File Inclusion, and cryptographic security concepts. The TryHackMe OWASP Top 10 2025 activity further supported understanding of access control, authentication, and logging and alerting concepts. The overall work also strengthened documentation and security reporting practices.

18. CONCLUSION

Week 4 – Task 12 provided practical exposure to common web application vulnerabilities and related security concepts through controlled laboratory exercises. The demonstrated activities show why secure input handling, authentication controls, cryptographic protection, safe operating-system interaction, file handling, and security monitoring are important components of application security.

Combining practical testing with secure development practices and appropriate defensive controls helps security professionals identify weaknesses and communicate meaningful remediation guidance. The evidence in this report is limited to the activities visibly demonstrated in the provided screenshots.

19. EVIDENCE SUMMARY

Figure	Evidence	Activity
Figure 1	DVWA Reflected XSS	Reflected XSS
Figure 2	DVWA SQL Injection	SQL Injection
Figure 3	DVWA Command Injection	Command Injection
Figure 4	DVWA Stored XSS	Stored XSS
Figure 5	DVWA Brute Force	Authentication Testing
Figure 6	DVWA File Inclusion	File Inclusion Testing
Figure 7	DVWA Cryptography Problems	Cryptographic Security
Figure 8	TryHackMe OWASP Top 10 2025	OWASP Security Learning

Evidence limitation: This report does not add CVE identifiers, CVSS scores, exploitation results, remediation results, SIEM dashboards, logs, or other activities that are not supported by the supplied evidence.